# 2

# Introduction to Python and the Field of Computational Statistics

**CHAPTER OBJECTIVES**

- Brief introduction and overview of Python software and how to install it on your machine.
- Why your focus should be on science and statistics first and foremost, not software.
- How to import packages into Python.
- How to compute *z*-scores in Python.
- Compute basic statistics such as means, medians, and standard deviations.
- How to build a dataframe or load external datasets into Python.
- Survey of essential linear and matrix algebra for statistics.

This chapter provides a brief introduction and overview to Python software with an emphasis, of course, on applications in statistics and data analysis. Unlike R and SPSS, two very popular statistical software packages used by statisticians, researchers, data scientists, and others, Python is a more general **computing language**, which, though useful for statistics, was not designed specifically for statistical applications such as R and SPSS (and SAS, STATA, and others). Python is used for **general computing purposes** by those generally in computer science and programming, and hence can be employed to generate a variety of computer programs and coding, the least of which is for statistical analyses. For many data-analytic tasks, software such as SPSS or Minitab or others provides more than sufficient capacity for analyzing data arising from the social, behavioral, medical, biological, and other sciences. Excel is also used by many for applications in finance, economics, and related areas. Having said all this, **why use Python at all then?**

The rationale for learning a programming language such as Python is similar to that of learning R. Aside from enjoyment value (some people like learning software for its own sake, the phrase "computer geek" did not evolve from nowhere!), it provides one with more flexibility and possibilities than "canned" software packages such as SPSS, SAS, etc. Furthermore, visualization capacities in R and Python far outweigh capacities in more traditional packages such as SPSS and Excel. When you learn a programming

*Applied Univariate, Bivariate, and Multivariate Statistics Using Python: A Beginner's Guide to Advanced Data Analysis*, First Edition. Daniel J. Denis.
© 2021 John Wiley & Sons, Inc. Published 2021 by John Wiley & Sons, Inc.

language such as Python, your creative capacity and range is greatly expanded and enriched, because you are essentially engaging in the same computational "basics" as even advanced programmers use to carry out extraordinary things, from producing sophisticated video computer games to launching a **SpaceX** vehicle. Learning a language such as Python is the starting point to doing such things, whereas the same cannot be said exactly for learning more canned packages. These latter programs already come "pre-packaged" to carry out statistical operations and functions and are based on an underlying language that simplifies things to get to the statistics more quickly. Python, on the other hand, is a more **primitive computing language** (by "primitive," we do not mean more "basic" or "elementary"), and hence when you are computing in Python, you are beginning more with the actual "sticks and stones" of computing (though Python, like R, also has packages for statistics). It gives you the opportunity to start with its most basic elements and building blocks. It is what computer scientists need to do regularly to program successfully, not simply use window GUI unsure of what the computer is actually doing behind the scenes. It is an exciting venture to embark upon because learning Python even at a very elementary level, as in this book, you enter the field of computer programming in general, a primer of sorts on your way to working in **computational statistics**. Of course, you will likely be spending most of your career computing statistics, and not programming per se, but the point is that with Python, you could theoretically do a lot more if you ever so choose. Learning Python to some degree also makes it easier to learn other languages should you choose to down the line. Analogous to learning one instrument such as the guitar, learning a second instrument such as the piano, though still difficult and challenging, becomes a bit easier because the "unifier" called "music" links both of them. If you learn Python, you can learn other languages as well (though some will be much more challenging).

*Contrary to SPSS or similar packages, when you learn Python, you are learning a very general-purpose computing language that can be used far beyond statistical applications. Though in this book our focus is on statistical modeling using Python, the sky is the limit in using Python for many programming needs you may come across in your career. Learning Python also allows you to generalize to other languages as well, such as Java. Though the languages are different, the learning curve to mastering other languages is greatly accelerated once you learn one of them.*

## 2.1 The Importance of Specializing in Statistics and Research, Not Python: Advice for Prioritizing Your Hierarchy

Having introduced this chapter and book featuring Python, I am now going to tell you why, as an aspiring scientist, you should probably not "specialize" in Python or any other software specifically. What I mean by this is that you should not claim Python as "your software" and close your doors to other programs, nor should you aim to necessarily master Python if your goal is that of being a scientist. Why not? Allow me to explain.

As a scientist, your primary obligation and specialization should be to your **field of study**. For example, if you are a biologist, then your focus should be on, well, biology. Perhaps you are passionate about the study of wolves. You thus busy yourself with the

study of such animals, focusing perhaps on their habitat, socialization patterns, and other fascinating things about these amazing animals. You do field trips to Yellowstone and follow a pack of wolves (not too closely, of course). In studying for your doctorate, however, you will likely need to conduct a study on wolves and analyze results quantitatively. You may even need to program an apparatus to study them in greater detail. The results you analyze will undoubtedly require the use and interpretation of statistics, which will thus require the use of software to conduct such analyses.

Hence, we already have a **hierarchy of specialization**, from biologist, to statistician, to computer programmer. Now, you are primarily a biologist, since, as mentioned previously, being a biologist is a full-time job. You can spend many lifetimes studying biology and only scratch the surface regarding what there is to know. However, without understanding statistics, you will not be able to interpret your scientific findings, and hence being part "applied statistician" to some degree is imperative. Otherwise excellent scientists who do not understand statistics are not excellent scientists at all, since they are unable to interpret research findings at a level of any critical depth, and are thus restricted to "trusting" to a great degree what is reported in published papers. The ideal combination is to be both an **applied scientist** and **applied statistician** (or data scientist, etc.), because it is only in having knowledge of both fields that you can confidently interpret research findings and all of its technicalities. For example, if one is unsure of what constitutes a **rigorous** vs. **non-rigorous** research design, one is simply unable to properly interpret empirical results and assign any meaning to them. Poor interpretations of experimental or non-experimental results may follow, leading to misinformed policy decisions or strategies you invoke in your clinical practice.

So what about computer programming? As is well known, computer science is a field unto itself. Computer scientists, among other things, busy themselves with writing and inventing new algorithms and developing new software. **They are specialists.** Our society is greatly influenced by such professionals, and as is true of the biologist, being a computer scientist is a full-time job. However, as a scientist and applied statistician, computer programming should correctly and most appropriately be ranked as your third priority in specialization. This is because beyond the specialized knowledge possessed by computer scientists, your goal, again as a biologist for example, is to use a programming language to address a biological problem, which, as mentioned, will often be analyzed using statistics. Hence, it is imperative to first understand the statistics you are using before applying them to the computer. This cannot be emphasized enough.

**If you do not understand the logic behind the statistics you are using, then it will matter little if you can plot down a bunch of computer code; and in fact, it can be dangerous to have a lot of computer knowledge but not understand the statistics you are computing. The old adage, "knowing just enough to be dangerous," is true in this regard.**

Hence, as a scientist, you are encouraged to prioritize science and statistics, and then dig for the computer code as you require it to address your scientific questions. In this respect, the computer code is the **machinery** you need to carry out what you have first intellectualized as part of your scientific reasoning process. Now, as you learn more statistics, the computational needs will likely also increase, but it is so important not to put computer programming before statistics, and statistics before your science, otherwise you run the risk of making very serious errors in interpretation of your substantive

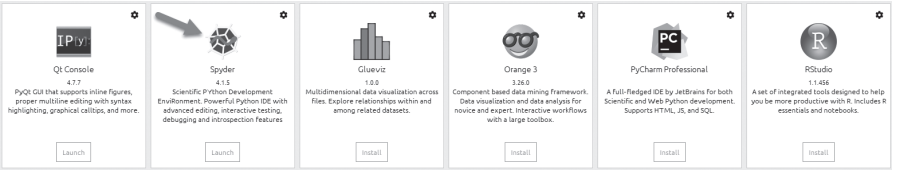
findings. Soon, the computer code and software fascination become a priority over the ignored weaknesses of your research design, which leads to faulty interpretation of data. It becomes a "gong show" of demonstrating how much computing, not science, you can do. The result? A bunch of statistical analyses on what is otherwise very weak science. Social scientists, in particular, are especially prone to this, to use statistics as a cover-up (often unknowingly) to an otherwise poor research design.

*It is imperative from the outset that you do not associate software knowledge with statistical knowledge. Learning software is a separate endeavor from understanding the statistics that underlie the software, or learning how to think statistically in general. As an applied scientist, you are encouraged to first specialize in your chosen field first and foremost, then focus on understanding how statistical thinking is used in your field, and then finally learn how to implement statistical computation in software.*

## 2.2 How to Obtain Python

With the earlier caveats in mind, we now proceed to introduce the software we will use for our analyses. **Python** is free software and is quite easily loaded onto your desktop or laptop computer. Most people who use Python access it through **Anaconda**, which consists of a number of Python computing environments. For instance, one may choose to use **IPython** or **Spyder** or other ways of accessing the software. You can access Anaconda via www.anaconda.com. From there, select "Get Started," and then "Install Anaconda Individual Edition" and download for your given operating system. In our case, we will download **Python 3.8 for Windows, 64-bit** (circled in the figure):


Once you have successfully downloaded the software, you can choose the **development environment** on which you wish to run Python. Many researchers and data scientists prefer to use the **Spyder** interface (shown in the image with the arrow pointing toward it), as it allows you to enter code and see output next to it in an adjacent window, almost analogous to using R in this regard. Others enjoy using **IPython**, while still others choose to use **Jupyter**. Whichever one you choose for our purpose and level of introduction is immaterial. They all run Python and the code presented in this book will work in any of them. Our goal for this book is not to survey computational environment possibilities, but instead to get on with learning how to use the software to perform statistical operations in analyzing one's data.




## 2.3 Python Packages

Similar to R software in which users can load packages to carry out specialized operations for the task at hand (e.g. one may require a particular package to carry out survival analyses, for instance), Python contains a series of packages that users can load into Python as needed. Common packages that you will see over and over include **Numpy** ("Numerical Python"), **Pandas**, **Matplotlib**, and many others. The good news is that the Anaconda distribution comes "pre-packaged" with many of these packages. Packages can be easily "imported" for use by the simple command **import**. For example, let's import **numpy**:

```
In [1]: import numpy
```

While the above will successfully import **numpy**, when we use **numpy**, we typically need to append the **numpy** name to the object we are working with. For example, suppose we wished to use **numpy** to sum a set of values for a variable. We will first give Python some data to work with and construct the variable (or vector) `x`:

```
x = numpy.array([0, 1, 2, 3, 4, 5])
numpy.sum(x)
Out[3]: 15
```

We see that Python has successfully added the numbers in the vector using `numpy.sum()`. However, instead of having to write out "numpy" each time, wouldn't it be easier to simply abbreviate it to a shorter label? This is exactly what most programmers do. That is, instead of importing **numpy** as itself, it is instead imported as "`np`":

```
import numpy as np
x = np.array([0, 1, 2, 3, 4, 5])
np.sum(x)
Out[6]: 15
```

We can see that other than identifying **numpy** as `np`, everything else in the earlier computation is the same. Hence, when you import packages, it is often useful to import them with abbreviations to make them easier to use and refer to later when you need them. Had we really wanted to, we could have imported **numpy** as:

```
import numpy as numpywhichisquitefascinating
numpywhichisquitefascinating.sum(x)
Out[8]: 15
```

Of course, doing the above would be a gigantic waste of our time. It is much easier to simply call it `np` and get on with more important things. As for **numpy**, as is true for

all Python packages, there are far too many functions to summarize here. Doing good data analysis means having a book or library (or Internet connection) available to you at all times so you can dig up code and functions as you require them. **Memorizing code, or attempting to learn all code, is a waste of your time and will be a goal forever unrealized. Learning how to problem-solve given novel data situations is the skill you actually need to learn.** Have you ever noticed that your mechanic does not always have the immediate answers, but knows where to look? Likewise, your veterinarian may not immediately know the problem with your dog, but a good veterinarian will know the path to pursue to finding the answer. Your approach to data analysis and computing should be that you may not have an immediate and complete grasp of the problem, but you know where you should look to find out, and you know the difference between being successful in the search vs. not. Recall from our previous discussion, however, that this should not be your goal regarding your science or knowledge of statistics. Understanding science and associated statistics is much more difficult to "dig up" as your knowledge requirements increase. Start with the solid scientific and statistical base, then dig up code as you require it.

For example, suppose you have created an array of data, which can be easily produced by `np.array()`:

```
x = np.array([1, 5, 8, 2, 7, 4])
Out[12]: array([1, 5, 8, 2, 7, 4])
```

Suppose you then want to know the value of a given element of the array. At first glance, you may not know how to retrieve this value. It is a relatively easy task in Python, but you may still not know how to do it. That's okay! That is why you have your Python books on your shelf or are equally able to access sources online. With even a bit of digging, you will find that you can easily access the fourth element by (counting "1" as zero):

```
x = np.array([1, 5, 8, 2, 7, 4])
x[4]
Out[16]: 7
```

Notice how easy that was! You may have not known beforehand how to index that number, but in seconds, you got what you needed from the manual or Internet, and now you are all set. You are not going to "memorize" how to index, but in time, you will simply remember instead of having to look it up. In addition, if you are coming to Python from R or similar software, you may already have an idea of how to index (or perform other functions), but need a bit more detail or specifics on how to get it done in Python. That is what the Internet or a source book is for. The process is one of "trial and error." Unless you are in a computer programming class, your instructor should not be testing you on how to index or to perform any other computer functions (to use this example). What they should be testing you on is whether you are able to access resources that allow you **independence** in using this or any other software. Therefore, in this book, while we could go on listing a catalogue of ways of doing almost an infinite number of things in Python, it is much easier to refer you to books already written that contain all the functions you may need as you progress in your career. Since this book is not a book on Python per se and our focus is on **understanding statistics and scientific principles**, we only survey a few of the most useful and in-demand

functions so you can get on with analyzing data quickly and efficiently. The rest will be up to you and your "digging skills" in accessing Python resources as you require them, remembering that each and every data analysis is unique and may require different functions.

As another example, suppose we needed to join or concatenate arrays in **numpy**. Suppose in addition to our `x` array, we also have a `y` array:

```
y = np.array([4, 6, 8, 2, 4, 1])
y
Out[20]: array([4, 6, 8, 2, 4, 1])
```

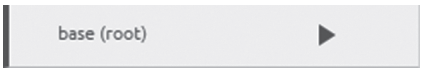
Now, let us concatenate the arrays using `np.concatenate()`:

```
np.concatenate([x, y])
Out[21]: array([1, 5, 8, 2, 7, 4, 4, 6, 8, 2, 4, 1])
```

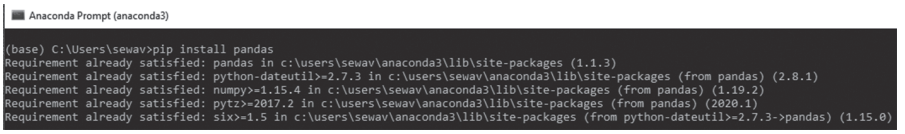
Notice that the `y` array now follows the `x` array.

## 2.4 Installing a New Package in Python

As mentioned, Anaconda comes already installed with most of the packages you will need for this book. However, at times you may need to install a new package. You can do this quite easily through **PIP**, which is a package-management system for installing new packages. In Anaconda, go to your base root directory:




Click on the arrow, and then select "Open terminal." Once in the new terminal, you can install a new package using **PIP**. For example, suppose we wanted to install the package **pandas**, we would enter in the prompt `pip install pandas`:




Of course, since pandas comes with Anaconda already, the system reports it as "already satisfied." However, later in the book (or in your future ventures using Python), you will need to install a package that does not already come with the Anaconda distribution. Once installed (as earlier), you can then return to your favorite Python environment and when needing the package, import it. Once you are back in Spyder (or the environment you prefer), you would code "`import pandas`" (or whatever package it is) and then it will be ready for use. We will demonstrate how this works as we proceed with computational examples in the book. If the earlier method of installation via PIP is not working for you, it may be because you are not accessing the correct folder on your computer and will need to do a bit of online digging and problem solving to rectify the issue (try Googling phrases such as "installing Python packages using PIP" to learn what may be going wrong).

## 2.5 Computing *z*-Scores in Python

Standardizing a distribution to *z*-**scores** means converting it from a raw distribution with a given mean and variance to a distribution with a mean of zero and variance of one. To obtain a *z*-score, we first subtract the mean from every observation (thereby "centering" the data). This produces a deviation of the form $x - \mu$. To get the *z*-score, we divide this deviation by the standard deviation, as follows:

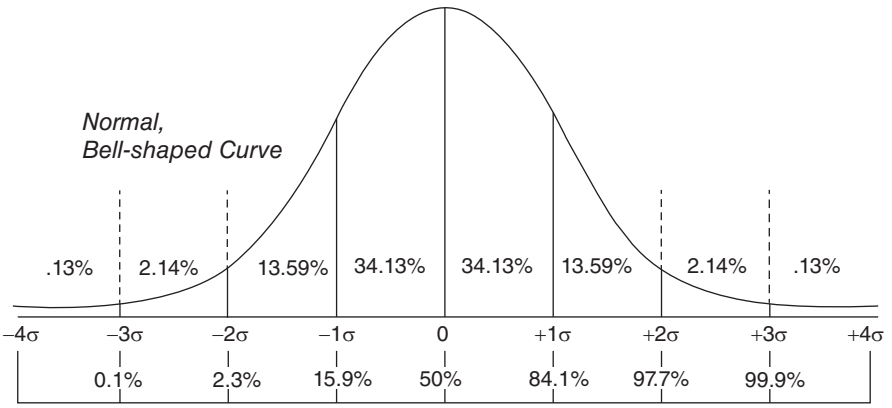
$$z = \frac{x - \mu}{\sigma}$$

Notice that the *z*-score is, in effect, generating a **ratio of one deviation to another**. In the numerator is the deviation of the given score from the mean, $x - \mu$, while in the denominator is the "average" or "standard" deviation, $\sigma$ . The question then becomes, **how big is the deviation for the given score relative to the average deviation**? This is essentially what a *z*-score is telling us. When you see the *z*-score as the ratio that it is, it begins to make more sense, and you do not simply associate with it being a "standardized" score and be satisfied with that "definition." Yes, the new mean will be 0 and the standard deviation 1, but the construction of the z-score has very much meaning in it. In general, whenever you look at formulas, always break them down into components and figure out what each is communicating. What is the formula really saying? Make no mistake, it is communicating a **concept.** It is much more than a bunch of symbols.

As an easy example, suppose one has a distribution of IQ scores measured on a sample or population of individuals. Suppose the mean is equal to 110 with standard deviation of 2. Suppose also that a given individual in the data scored 114. What is that person's *z*-score? To obtain the *z*-score, we calculate:

$$z = \frac{x - \mu}{\sigma} = \frac{114 - 110}{2} = \frac{4}{2} = 2$$

We can see for a score of 114 that the individual lies 2 standard deviations above the mean. If the distribution is normal, then, as shown, we can conclude that the given individual scores higher than approximately 97.7% of the given IQ distribution (i.e. $+2\sigma$ ):



We can easily compute $z$-scores in Python. To demonstrate, we will first load a data set that we will refer to often throughout the book. On this fictitious data achievement scores in mathematics were recorded as a function of which teacher and textbook students were paired with (we will describe the details of how we build the dataframe a bit later and print only a few cases of the output here):

```
data = {'ac' : [70, 67, 65, 75, 76, 73, 69, 68, 70, 76, 77, 75, 85,
86, 85, 76, 75, 73, 95, 94, 89, 94, 93, 91],
'teach' : [1, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 2, 3, 3, 3, 3, 3, 3, 4,
4, 4, 4, 4, 4],
'text' : [1, 1, 1, 2, 2, 2, 1, 1, 1, 2, 2, 2, 1, 1, 1, 2, 2, 2, 1,
1, 1, 2, 2, 2]}
import pandas as pd
df = pd.DataFrame(data)
df
Out[82]:
    ac  teach  text
0   70      1     1
1   67      1     1
2   65      1     1
3   75      1     2
4   76      1     2
5   73      1     2
6   69      2     1
7   68      2     1
8   70      2     1
9   76      2     2
10  77      2     2
```

Suppose we would like to transform our `ac` variable from the achievement data into one of $z$-scores. Let us begin with a computation that will compute them "manually," meaning that we are going to tell Python exactly what to do to get the scores. That is, we are going to compute a function that will compute $z$-scores for us:

```
import statistics
ac = df['ac']
z_numerator = (ac - statistics.mean(ac))
z_denominator = statistics.stdev(ac)
z = z_numerator/z_denominator

z
Out[33]:
0    -0.937148
1    -1.248091
2    -1.455387
3    -0.418910
4    -0.315262
5    -0.626205
```

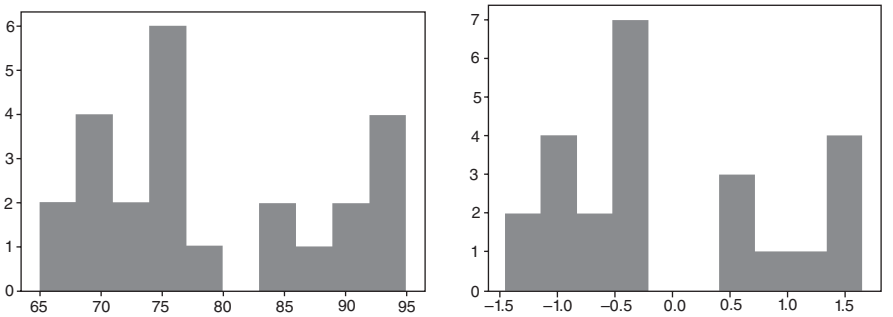
Note in our computation we first took ac scores then subtracted the mean to give us z_numerator, then computed z_denominator using the **statistics** module statistics.stdev to get the standard deviation. The *z*-score is then computed as the numerator divided by the denominator, z_numerator/z_denominator. Throughout the book we present computations for various measures, but will not always describe the computation in detail as we did here. With practice, you will begin to see the code and be able to decipher what is being computed. This is true of more advanced computing as well; always investigate the code for ideas and hints as to what is being computed.

Now, as you may imagine, computing statistics the long way is inefficient and very time-consuming. We can often do much better by using **packages** directly. We now generate the *z*-scores using **scipy** by first importing stats:

```
from scipy import stats
ac = df['ac']
stats.zscore(ac, axis=0, ddof=1)
Out[28]:
array([-0.93714826, -1.24809146, -1.45538693, -0.41890959, -0.31526186,
       -0.62620506, -1.04079599, -1.14444373, -0.93714826, -0.31526186,
       -0.21161412, -0.41890959,  0.61756775,  0.72121548,  0.61756775,
       -0.31526186, -0.41890959, -0.62620506,  1.65404508,  1.55039735,
        1.03215868,  1.55039735,  1.44674962,  1.23945415])
```

Note how the *z*-score is requested. The function zscore requests zscores from the variable ac in the dataframe df. Had we wanted to compute *z*-scores for a different variable from the dataframe df, then instead of ['ac'] we would have specified whatever variable we wished. Next, let us generate a histogram of ac raw scores and then one on our newly computed *z*-score distribution. For this, we use matplotlib.pyplot:

```
import matplotlib.pyplot as plt
ac = df['ac']
z = stats.zscore(ac, axis=0, ddof=1)
ac_hist = plt.hist(ac)
ac.z_hist = plt.hist(z)
```




Notice that the plots, whether in raw ac scores or *z*-scores, have the same distribution (the scaling and grouping of intervals along the *x*-axis makes them look slightly

different). **Converting to z-scores does not change the shape of the original distribution.** That is, the fact that we linearly transformed them to *z*-scores does not change its shape. The computation $z = (x - \mu)/\sigma$ simply generates a new mean (of zero) and standard deviation of one. This is an example of a linear transformation. Had we transformed this into something that is **non-linear**, then the distribution would have changed. Often students will assume that transforming to *z*-scores will "normalize" an otherwise skewed or abnormal distribution, but this is not the case. If you have a skewed distribution and wish it to be more normal in shape, then a non-linear transformation may be called for, such as taking **squares** or **square roots**. A logarithmic transformation on data may also prove useful at times.

We can also easily obtain measures of **skewness** and **kurtosis**, these coming from the package **scipy**:

```
import scipy as sp
sp.stats.skew(ac)
Out[190]: 0.3882748054677566

sp.stats.kurtosis(ac)
Out[189]: -1.2190774612249433
```

The **skewness** value of 0.388 indicates a slight **positive skew** (greater than 0), while the **kurtosis** value of –1.219 (less than 0) indicates that the distribution's tails are not so pronounced, or, roughly equivalent, that the distribution is a bit flatter than one would have with no kurtosis (the so-called **mesokurtic** distribution). This latter definition should be used with caution (see DeCarlo, 1997), but is often a useful crude and "working rule" to define kurtosis (one can use scipy's `kurtosistest()` to evaluate kurtosis inferentially). While measures of skewness and kurtosis are sometimes useful and meaningful in some work, in data analysis, graphical representations of data usually suffice. For the most part, you need not worry too much about numerical measures of skewness and kurtosis. Visualization is much more powerful and useful in this regard to detect deviations from ideal shapes.

*A z-score is a comparison of one deviation to another. The deviation in the numerator is the observed deviation for the given observation from the mean of the data. The deviation in the denominator is the average deviation in the data, the so-called "standard" deviation. While converting a distribution of raw scores to standard scores generates a new distribution with mean 0 and variance 1, it does not "normalize" the distribution. Hence, the distribution of z-scores will have the same shape as the original distribution. It will not suddenly become normal due to the transformation to z-scores.*

## 2.6 Building a Dataframe in Python: And Computing Some Statistical Functions

There are many ways of building and accessing data in Python. We will survey a few of these as we progress through the book. For now, we wish to demonstrate how a dataframe can be built from scratch by first entering the data right into our work area and

then obtaining a few statistics on the dataframe to get us going. In fact, we have already done this just above when we surveyed the achievement data. Recall that we had laid out of our data on each variable and then joined it into a dataframe:

```
data = {'ac' : [70, 67, 65, 75, 76, 73, 69, 68, 70, 76, 77, 75, 85,
86, 85, 76, 75, 73, 95, 94, 89, 94, 93, 91],
'teach' : [1, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 2, 3, 3, 3, 3, 3, 3, 4,
4, 4, 4, 4, 4],
'text' : [1, 1, 1, 2, 2, 2, 1, 1, 1, 2, 2, 2, 1, 1, 1, 2, 2, 2, 1,
1, 1, 2, 2, 2]}
import pandas as pd
df = pd.DataFrame(data)
df
Out[82]:
    ac  teach  text
0   70      1     1
1   67      1     1
2   65      1     1
```

To recap what we did earlier, the line `df = pd.DataFrame(data)` is what is creating the dataframe from the set of numbers contained in `data`. Our new dataframe is thus `df`. As mentioned, we will revisit this data later in the book when we use it for statistical models. For now, we simply wish to demonstrate a few of the computations we can perform on the data file to help us learn more about it and to demonstrate the power of Python. Let us first demonstrate how we can compute a few functions from the "ground-up" so to speak. For example, suppose we wish to compute the mean of `ac`. We have already seen how we can extract the variable `ac`. We print only a few cases here:

```
df['ac']
Out[107]:
0     70
1     67
2     65
3     75
```

Now let us compute the arithmetic mean of `ac`. We can do this easily by giving the object a new name:

```
ac = df['ac']
mean = sum(ac)/len(ac)
mean
Out[111]: 79.04166666666667
```

In the above, we summed all observations in `ac` by `sum(ac)` and then divided this sum by the length of `ac`, denoted `len(ac)`. The length is the number of values that went into the sum. Hence, what the above is computing is simply

$$\bar{y} = \frac{\sum_{i=1}^{n} y_i}{n}$$

The mean is 79.04. Note that this is the **arithmetic mean**. There are other means, such as the **harmonic** and **geometric** means that are beyond the scope of this book but are sometimes useful in data analysis in particular contexts. The point for now is that when you hear "mean" do not necessarily assume it is the arithmetic mean. It usually will be, but not always. Of course, we could have found the mean using one of Python's built-in functions. By importing **statistics**, we could have also computed the mean by using `statistics.mean()`:

```
import statistics
mean = statistics.mean(ac)
mean

Out[115]: 79.04166666666667
```

We could have also just as easily used **numpy** to compute the mean:

```
import numpy as np
np.mean(ac)

Out[116]: 79.04166666666667
```

Obtaining a **standard deviation** is just as easy. A standard deviation, by definition, is the square root of the variance, computed as follows:

$$s = \sqrt{\frac{\sum_{i=1}^{n}(x_i - \bar{x})^2}{n-1}}$$

Notice we are taking the sum of squared deviations from the mean for each score and then dividing by the number of observations that went into the sum. For the current computation, since we wish to use the sample standard deviation as an **unbiased estimator** of the population standard deviation, we lose a **degree of freedom** in the denominator. If we wished to only compute the standard deviation on the sample and not use it as an estimator of the population parameter, then we could use the following:

$$s = \sqrt{\frac{\sum_{i=1}^{n}(x_i - \bar{x})^2}{n}}$$

Usually, however, since we typically do not have the population standard deviation at our disposal and wish to use the sample standard deviation as an estimator, $n-1$ is used. For a proof as to why $n-1$ generates an unbiased estimator, see Hays (1994) or virtually any book on mathematical statistics. For a less technical explanation and one that is a bit more intuitive, see Denis (2021).

The variance of the data can be computed via `np.var(ac) = 89.20`. Now let us get the standard deviation. We will use the **math** module for this, specifically the `math.sqrt()` function:

```
import math
math.sqrt(89.20)

Out[147]: 9.444575162494075
```

To get the population standard deviation, we compute:

```
sd = statistics.pstdev(ac)
sd
```

```
Out[162]: 9.444924415908378
```

Hence, we can say that, on average, scores deviate from the mean by approximately 9.44 scores in this distribution. If we wanted the **median** of our variable, the value that divides the data into two equal parts, we could also easily compute this:

```
statistics.median(ac)
Out[157]: 76.0
```

Note that the median is a bit less than the mean of 79, indicating that a few of the higher-end scores are pulling the distribution up a bit toward the right side of the distribution. This is so because the mean is **sensitive** to extreme scores whereas the median is not. In cases where you have a highly skewed distribution especially, computing the median rather than the mean provides a more **resistant** (i.e. resistant to extreme scores) measure of central tendency. We could also try the mode for `ac`, but this is what we might get:

```
statistics.mode(ac)
StatisticsError: no unique mode; found 2 equally common values
```

This is hardly an "error," as it simply means there is no **unique mode** in the data. However, one could just as easily, if not somewhat informally, say that there are more than two modes. So long as you are communicating this clearly to your reader, it does not really matter how you say it, so long as the message gets through.

*Means and standard deviations are often useful for summarizing distributional characteristics of variables. However, both are sensitive to extreme or outlying scores, and hence at times more resistant measures such as the median, range and interquartile range are sometimes preferred.*

## 2.7 Importing a .txt or .csv File

Usually, importing and reading a text file is relatively straightforward. However, sometimes it may require some adjustment. Here is a problem you may run into. Suppose you would like to load a text or .csv file, but that it appears as follows when you load it (only a few observations of the entire data set are shown). We are correctly using `pd.read_csv` to attempt to read it, but we get the following, when we should be getting simply numbers under the headers `verbal`, `quant`, `analytic` and `group`:

```
import pandas as pd
iq_data = pd.read_csv('iq.data.txt')
```

```
iq_data
```

```
Out[295]:
    Unnamed: 0 verbal quant analytic group
0            0    56.00\t56.00\t59.00\t.00
1            1    59.00\t42.00\t54.00\t.00
2            2    62.00\t43.00\t52.00\t.00
```

The file is coming out this way because Python is not recognizing the spaces between numbers. This can be fixed easily by the following, in which we specify a `delimiter`:

```
dataset=pd.read_csv("iq.data.txt",delimiter="\t")


dataset
Out[316]:
verbal quant analytic group
56.0 56.0 59.0 0.0
59.0 42.0 54.0 0.0
62.0 43.0 52.0 0.0
```

We see that now the file is read correctly. In general, if a file is not being imported properly, it may require a bit of digging online or through Python sources to figure out why and then fix things. Never underestimate how long it can take just to get a file working correctly. Data tasks like this can easily consume a lot of work. In data science lingo, ensuring that your data is organized properly is often referred to as "**tidying**" a data set (see Wickham and Grolemund, 2017, for more details).

## 2.8 Loading Data into Python

Entering data directly into Python is fine as we did earlier on the achievement data, and for small projects, may be a sufficient way to get your data into Python. Sometimes, entering data by hand is a nice idea, especially if the data set is small, because it allows you to screen the data as you enter it. For example, if you are entering ages of people, and you come across an age of 200, you will immediately recognize that something is amiss. Often, however, data come in very large sizes and we are best loading it from **external files**. Sometimes, we would like to load it from R software, since R has many **prepackaged** data sets. One nice data feature of Python is the ability to access data sets in R via `get_rdataset()`. Below we access the **iris** data using **seaborn**, which is a historic data set containing the lengths and widths of sepals and petals. As shown, we request the data from `sns` and then request a few cases:

```
import seaborn as sns
iris = sns.load_dataset('iris')

iris.head()
Out[133]:
   sepal_length  sepal_width  petal_length  petal_width species
0           5.1          3.5           1.4          0.2  setosa
1           4.9          3.0           1.4          0.2  setosa
```

```
2            4.7          3.2          1.3          0.2  setosa
3            4.6          3.1          1.5          0.2  setosa
4            5.0          3.6          1.4          0.2  setosa
```

## 2.9 Creating Random Data in Python

Instead of our own data, sometimes we would like to create random data, especially for the sake of demonstrating statistical principles. It provides a powerful tool for teaching. What is random data generated by a computer? It is not what you may think. **No computer can actually generate truly random data**. It is best called "**pseudo-random**" since it is still based on an algorithm, which is a deterministic way of generating data no matter how much we would like to believe it to be probabilistic. Hence, when we ask any computer to generate random data, it is a bit different from flipping coins, where one might say a true random process is present (although one could argue a higher power knows the deterministic algorithm at work in the coin flips, but we as mortal beings certainly do not).

We can use `np.random.randn()` to generate the random numbers we desire. We will do so for 100 random digits (printing only a few cases):

```
import numpy as np
np.random.randn(100)
Out[54]:
array([ 0.02269355, -0.07788592,  0.45105672, -1.19752485,  0.87797989,
        0.46153428, -0.43629477,  1.28920105, -0.74609082, -0.02370795,
       -0.13875894,  0.16783418,  0.68196274, -0.07277099,  0.81438243,
        1.34040424, -1.84640245, -0.30100139, -0.21200374,  0.99749304,
```

## 2.10 Exploring Mathematics in Python

Not surprisingly, Python is able to compute just about any mathematical function we might desire. Since statistics is built on the foundations of mathematics and philosophy, surveying some of these mathematical capabilities is well worth our time. In some cases (such as in coursework for good practice), you may wish to build up the programs yourself so that you become completely aware of what the function is actually doing. In other cases, simply obtaining the function may be enough. For example, we can easily obtain the constants for $e$ and $\pi$ from the **math** module:

```
import math
math.e
Out[154]: 2.718281828459045

math.pi
Out[155]: 3.141592653589793
```

The constant $e$ is **Euler's number**, found virtually everywhere in mathematics, statistics, and science more generally, whereas $\pi$ is another constant found in innumerable places, well-known for being the ratio of the circumference to diameter of a circle.

These are **constants** that find themselves in many key formulas and functions in even so-called elementary statistics. For example, the equation for the normal distribution features both of them:

$$f(x_i,\mu,\sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-(x_i-\mu)^2/2\sigma^2}$$

where, in the equation, $\mu$ is the population mean for the given density, $\sigma^2$ is the population variance, $\pi$ is the "pi" constant we just mentioned, equal to approximately 3.14, and $e$ is the other constant, equal to approximately 2.71. The value for $x_i$ is a given value of the independent variable, assumed to be a real number. The values for $\pi$ and $e$ are regarded as constants because they remain the same regardless of the values of the other "inputs" to the function rule. That is, we can essentially imagine an infinite number of normal distributions as defined by the above rule, but all will have $\pi$ and $e$ in them, and these numbers will not change. They are "constants" across variations of the function rule. This is in contrast to $x_i$, which is a **variable** in the expression, as its value will change depending on what value we input for it.

*The normal distribution is a family of distributions defined by a function rule. There are literally an infinite number of normal distributions possible, each defined by parameters $\mu$ and $\sigma^2$. Exact normal distributions will never appear in nature. Since they are defined by a function rule first and foremost, the best empirical data will do is approximate a normal density, but it will never exactly mirror one.*

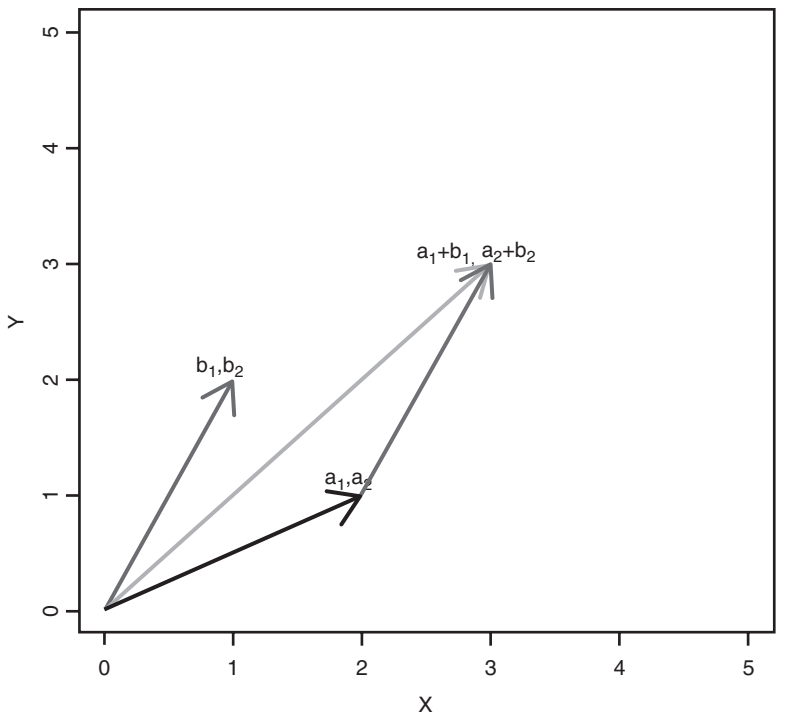
## 2.11 Linear and Matrix Algebra in Python: Mechanics of Statistical Analyses

As we began discussing in the previous chapter, underlying every statistical analysis you or I perform with software is a wealth of **mathematical** and **philosophical** foundations. For example, when we conduct a **principal components analysis**, which is a **multivariate** statistical procedure, what we are seeing in software output is but the surface to a whole underground of mathematical and philosophical structures. This, again, is one of the dangers of conducting analyses one may not understand at a deeper level. It becomes much too easy to draw research and scientific conclusions from software output that are not defensible since the underlying mathematics or philosophy may simply not allow for such conclusions to be drawn. The software does not "know" how a logical process functions toward drawing a scientific conclusion. The computer simply computes what you tell it to, and it does so extremely reliably and very fast. To a computer, a complex statistical analysis is a series of computations and nothing more. It is not aware of how you are using such computations or what conclusions you hope to draw from them. For example, the computer knows how to compute an arithmetic mean, but it does not "understand" the advantages or limitations of using the mean as a measure of central tendency. It is up to you to know that and to use the computations generated by software in an intelligent manner. **As a scientist, you should be very familiar with the underpinnings of the tools you work with**. You may not appreciate why this is so important at first glance, but as you become more experienced, you will soon realize that not understanding the mathematics and

essential philosophy "behind the scenes" of your software output can have rather serious implications. **Always strive to understand the method you are using at the deepest level you can before using it in Python or any other software**. **This includes understanding both the mathematical and philosophical issues underlying the method you are using**.

Linear and matrix algebra is based on vectors and matrices. A **vector** in mathematics is usually defined as a **line segment having magnitude and direction**. This is the classic **physics** definition of the term and it is not difficult to see why when we consider physical applications. For example, vectors help aircraft determine desired direction and arrival points. Hence, when air traffic controllers are communicating navigation information to pilots, precise vector information provides reliable coordinates. In **computer science**, a vector may also simply be considered as a listing of numbers. For example, the numbers (5, 8, 9, 10) can be considered a vector. Hence, in this computer science sense of the word, vectors do not necessarily have to have magnitude or direction as in the sense of an implied physical force in physics. Via this simple example, one immediately gets the correct sense that many terms in mathematics, even though seemingly rigorously defined, can still have slightly different meanings depending on the given area of application. This is again why attempting to understand definitions outside of their context can be challenging, such that if you attempted to memorize a definition for vectors, you may not be aware when it is used in a slightly different context than the definition you originally learned. Definitions, especially in statistics, do their best to capture the essence of the given concept or construct, but only experience with the concept will encourage and foster a deeper understanding.

As an example of two vectors that have been added, consider the following:



Each tip of a vector represents a given point in the plane. For instance, the point $a_1, a_2$ corresponds to the point $x = 2$ and $y = 1$. Though the addition of the two vectors need not really concern us, it is enough to know for now that two vectors can be added, multiplied, and subject to many other mathematical operations. The vectors shown are in two-dimensional space, but mathematically there is no limit to how much we can generalize the dimensions of vectors. Mathematically, we could be working in 100 dimensions if we chose and the principles behind vector operations will remain the same. Mathematicians can work in virtually as many dimensions as they choose, since their abstract work does not necessarily need to correlate with the empirical world or "real" objects. Pure (not applied) mathematics is (or can be considered to be) an **abstract** system.

A **matrix** in linear algebra can be defined as many vectors "glued" together, such that the matrix now has more than a single column or single row. An example of a matrix is the following:

$$\mathbf{A} = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{bmatrix} = \begin{pmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{pmatrix} = \left(a_{ij}\right) \in \mathbb{R}^{m \times n}$$

Notice the matrix has $m$ rows and $n$ columns, and is typically written with square or round brackets as above. Many advanced statistical procedures operate directly on vectors and matrices. For instance, **principal components analysis** as well as **factor analysis** typically use a **covariance** or **correlation matrix** as input to the analysis. For example, the following is a covariance matrix:

$$K_{XX} = \begin{bmatrix} E\left[\left(X_1 - E[X_1]\right)\left(X_1 - E[X_1]\right)\right] & E\left[\left(X_1 - E[X_1]\right)\left(X_2 - E[X_2]\right)\right] & \cdots & E\left[\left(X_1 - E[X_1]\right)\left(X_n - E[X_n]\right)\right] \\ E\left[\left(X_2 - E[X_2]\right)\left(X_1 - E[\mathbf{X_1}]\right)\right] & E\left[\left(X_2 - E[X_2]\right)\left(X_2 - E[X_2]\right)\right] & \cdots & E\left[\left(X_2 - E[X_2]\right)\left(X_n - E[X_n]\right)\right] \\ \vdots & \vdots & \ddots & \vdots \\ E\left[\left(X_n - E[X_n]\right)\left(X_1 - E[X_1]\right)\right] & E\left[\left(X_{\boldsymbol{n}} - E[X_n]\right)\left(X_2 - E[X_2]\right)\right] & \cdots & E\left[\left(X_n - E[X_n]\right)\left(X_n - E[X_n]\right)\right] \end{bmatrix}$$

Along the **main diagonal** (top left to bottom right) of the matrix are variances, whereas along the **off-diagonal** (i.e. above or below the main diagonal) are covariances. The notation "E" denotes **expectation**, which is a long-run weighted average. Expectations are used heavily in theoretical statistics and probability to give us an idea of what is being estimated in the long run by a sample quantity. If we standardize the covariance matrix, we obtain the correlation matrix:

$$\operatorname{corr}(\mathbf{X}) = \begin{bmatrix} 1 & \dfrac{E\left[\left(X_1 - \mu_1\right)\left(X_2 - \mu_2\right)\right]}{\sigma\left(X_1\right)\sigma\left(X_2\right)} & \cdots & \dfrac{E\left[\left(X_1 - \mu_1\right)\left(X_n - \mu_n\right)\right]}{\sigma\left(X_1\right)\sigma\left(X_n\right)} \\ \dfrac{E\left[\left(X_2 - \mu_2\right)\left(X_1 - \mu_1\right)\right]}{\sigma\left(X_2\right)\sigma\left(X_1\right)} & 1 & \cdots & \dfrac{E\left[\left(X_2 - \mu_2\right)\left(X_n - \mu_n\right)\right]}{\sigma\left(X_2\right)\sigma\left(X_n\right)} \\ \vdots & \vdots & \ddots & \vdots \\ \dfrac{E\left[\left(X_n - \mu_n\right)\left(X_1 - \mu_1\right)\right]}{\sigma\left(X_n\right)\sigma\left(X_1\right)} & \dfrac{E\left[\left(\boldsymbol{X}_n - \mu_n\right)\left(X_2 - \mu_2\right)\right]}{\sigma\left(X_n\right)\sigma\left(X_2\right)} & \cdots & 1 \end{bmatrix}$$

Notice that to get the correlation matrix from the covariance matrix, we divide each entry by the product of standard deviations for $X_1$ and $X_2$, where $\sigma$ denotes the standard deviation and $\sigma(X_1)\sigma(X_2)$ is the product of standard deviations for $X_1$ and $X_2$.

We can build a covariance matrix in Python quite easily. Let us first build up some fictitious data on two variables by generating a dataframe using `pd.Dataframe():`

```
cov_matrix = pd.DataFrame([(10, 20), (5, 15), (20, 12), (8, 17)],
columns=['var1','var2'])

cov_matrix
Out[189]:
   var1  var2
0    10    20
1     5    15
2    20    12
3     8    17
```

The above denotes the scores on `var1` to be 10, 5, 20 and 8, and so on for `var2`. We have named the object `cov_matrix`, even though it is not a matrix yet, but simply a listing of two variables. To get the matrix, we attach `.cov()` to it:

```
cov_matrix.cov()
Out[190]:
            var1        var2
var1   42.250000 -12.333333
var2  -12.333333  11.333333
```

For reasons we will discuss in the following chapter, we often prefer the **correlation matrix** to the covariance matrix for many analyses. As mentioned, the correlation matrix is the standardized version of the covariance matrix. We can easily standardize the covariance matrix by using `.corr()` instead of `.cov()`:

```
cov_matrix.corr()
Out[196]:
          var1      var2
var1  1.000000 -0.563622
var2 -0.563622  1.000000
```

We can see from the matrix that the correlation between `var1` and `var2` is equal to –0.563622, meaning that the correlation between the two variables is moderately negative. We also see in the main diagonal from top left to bottom right that there are values of 1.000000. This is as a result of the correlation of a variable with itself being perfect and positive. Again, we delay our discussion of covariance and correlation to the following chapter, where we unpack the nature of correlation and explain why the correlation coefficient is bounded by values of –1.0 and 1.0.

### 2.11.1 Operations on Matrices

Earlier we constructed a covariance and correlation matrix, but the sky is the limit regarding the types of matrices one can build in Python or other software. For instance, the following is a matrix made up entirely of zeroes having four rows and four columns:

```
import numpy as np
np.zeros((4, 4))
Out[287]:
array([[0., 0., 0., 0.],
       [0., 0., 0., 0.],
       [0., 0., 0., 0.],
       [0., 0., 0., 0.]])
```

We could have built the same matrix using the following as well by specifically designating the matrix as `numpy.empty()`. An **empty matrix** is one with zeros within the entire matrix:

```
import numpy as np
np.empty((4, 4))
Out[289]:
array([[0., 0., 0., 0.],
       [0., 0., 0., 0.],
       [0., 0., 0., 0.],
       [0., 0., 0., 0.]])
```

We could also easily build a matrix having values of 1.0 everywhere. For example, the following matrix has four rows and two columns with values of 1.0. Notice that instead of `numpy.zeroes()` (which would generate zeros) we are using `numpy.ones()`:

```
import numpy as np
np.ones((4, 2))
Out[290]:
array([[1., 1.],
       [1., 1.],
       [1., 1.],
       [1., 1.]])
```

When we have different values in our matrix, we can build it by specifying each of them. The following is a $2 \times 2$ matrix having values of 1, 2, 3, 4:

```
import numpy as np
A = np.matrix('1 2; 3 4')

A
Out[160]:
matrix([[1, 2],
        [3, 4]])
```

We could have also used the `np.array()` function to build the same matrix. We name this matrix B to distinguish it from the above, even though it contains the same elements:

```
import numpy as np
B = np.array([[1, 2],
[3, 4]])
B
```

```
Out[165]:
array([[1, 2],
       [3, 4]])
```

The **transpose** of a matrix is defined by exchanging rows for columns and columns for rows. That is, if our original matrix is

$$\mathbf{B} = \begin{bmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{bmatrix}$$

then the transpose is defined by $\mathbf{B}'$:

$$\mathbf{B}' = \begin{bmatrix} b_{11} & b_{21} \\ b_{12} & b_{22} \end{bmatrix}$$

Notice that the first row, $b_{11} \quad b_{12}$, is now the first column, and likewise for the second row. In Python, we can easily compute the transpose by again a simple extension to `np`, this time `np.transpose()`:

```
import numpy as np
np.transpose(B)
Out[178]:
array([[1, 3],
       [2, 4]])
```

Notice that the rows of the old matrix are now the columns of the new matrix.

Another important measure as it concerns matrices is the **trace of a matrix**, which consists of summing the values along the main diagonal of the matrix. For example, the trace of the matrix B transpose that we just computed is equal to 5, since the values along the main diagonal (top left to bottom right) consist of values 1 and 4 for a total of $1 + 4 = 5$. More generally, the trace for a $3 \times 3$ matrix

$$\mathbf{A} = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix}$$

is equal to $a_{11} + a_{22} + a_{33}$. The trace of **non-square matrices** is not defined since for non-square matrices there exists no **principal diagonal** (i.e. no "top left to bottom right" idea). We can easily compute the trace in Python by this time adding the extension of `matrix.trace` on `np`. We compute this for our original matrix B:

```
import numpy as np
np.matrix.trace(B)
Out[198]: 5
```

We can also request the elements along the main diagonal specifically:

```
import numpy as np
np.matrix.diagonal(B)
Out[201]: array([1, 4])
```

So why bother computing something called the trace? Computing the trace is especially important when computing a variety of multivariate test statistics, as we will see later in the book when we consider multivariate procedures. Its relevancy is also especially apparent when working with principal components analysis, as **the sum of variances of all variables is equal to the trace of the covariance matrix**. As we will see, this sum will also equal the **sum of eigenvalues** across the matrix. Hence, traces of matrices are used quite regularly in the communication of multivariate results. In the context of matrix theory, we now briefly survey the very important concepts of eigenvalues and eigenvectors.

### 2.11.2 Eigenvalues and Eigenvectors

At first glance, multivariate procedures can seem formidable. Though they are rather complex, there is a unifying component to them that remarkably encompasses much of their complexity, and that is in the mathematical objects of **eigenvalues** and **eigenvectors**. In most univariate and multivariate procedures, the essential computation that is occurring "behind the scenes" is a computation of eigenvalues and eigenvectors. Though a deep investigation into these concepts is well beyond the scope of this book, we can nonetheless get the main ideas across by a simple survey of them. Be forewarned that they will make much more sense when we survey **principal components analysis** and other dimension-reduction techniques later in the book. Or at minimum, you will see how they are used in applied analysis and situate them in some context other than abstract mathematics.

Suppose we have a square matrix of dimension $n \cdot n$. We will call this matrix $\mathbf{A}$. Now, it is a result of mathematics that the following equality is true with regards to matrix $\mathbf{A}$:

$$\mathbf{Ax} = \lambda\mathbf{x}$$

In this equation, $\mathbf{x}$ is a vector and $\lambda$ is a scalar, that is, an ordinary real number. In English, what the above equality says is this: Multiplying vector $\mathbf{x}$ by the matrix $\mathbf{A}$ generates the same result as multiplying vector $\mathbf{x}$ by the scalar $\lambda$. The scalar $\lambda$ is called an **eigenvalue** of the matrix $\mathbf{A}$ and the vector $\mathbf{x}$ is called an **eigenvector** associated with $\lambda$. The equivalency $\mathbf{Ax} = \lambda\mathbf{x}$ when worked on slightly algebraically can be written as follows:

$$\mathbf{Ax} - \lambda\mathbf{x} = \mathbf{0}$$
$$(\mathbf{A} - \lambda\mathbf{I})\mathbf{x} = \mathbf{0}$$

If the determinant of this expression does not equal 0, that is, if $|\mathbf{A} - \lambda\mathbf{I}| \neq \mathbf{0}$, then this also implies that $(\mathbf{A} - \lambda\mathbf{I})$ has an inverse, which further implies that the only vector $\mathbf{x}$ that will provide a solution is that of $\mathbf{x} = \mathbf{0}$. The solution $\mathbf{x} = \mathbf{0}$ is known as the trivial solution. To obtain a solution that is not equal to 0, $|\mathbf{A} - \lambda\mathbf{I}|$ is set to 0 and then values for $\lambda$ are found that, when substituted back into $(\mathbf{A} - \lambda\mathbf{I})\mathbf{x} = \mathbf{0}$, give a solution for $\mathbf{x}$. The equation $|\mathbf{A} - \lambda\mathbf{I}| = \mathbf{0}$ is known as the **characteristic equation**, and for a square matrix, that is, one that has $n$ rows and $n$ columns, the characteristic equation will have a total of $n$ roots. But what is a root? A **root** is where the function intersects the $x$-axis, a value that once inserted into the equation will make the value of the equation

equal to 0. Recall that to compute the roots of a quadratic equation, we can use the quadratic formula:

$$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

where the values of $a, b$, and $c$ are from the general quadratic equation, $ax^2 + bx + c = 0$. These roots in the context of eigenanalysis are referred to as **eigenvalues**. For the $n \cdot n$ matrix, the characteristic equation will have $\lambda_1, \lambda_2, ..., \lambda_n$ eigenvalues, where some values may be the same.

We can obtain the eigenvalues and eigenvectors of a matrix in Python quite easily. For example, the eigenvalues and eigenvectors of the matrix B can be obtained from **linalg** in **numpy** using the extension `.eig()`:

```
B = np.array([[1, 2], [3, 4]])
B
Out[7]:
array([[1, 2],
       [3, 4]])

results = np.linalg.eig(B)

results
Out[46]:
(array([-0.37228132,  5.37228132]), array([[-0.82456484, -0.41597356],
       [ 0.56576746, -0.90937671]]))
```

The first array reports the eigenvalues for the 2 × 2 matrix. The eigenvalues are thus –0.37 and 5.37, while the corresponding eigenvectors are –0.82, 0.56 and –0.41, –0.90, respectively. Again, such computations will have much more meaning when we consider them later in the book in the context of actual statistical procedures such as **discriminant analysis** and **principal components analysis**.

## Review Exercises

1. Explore and discuss why specializing in your field of study first, then statistics second, and then software computations third, should be your priority. What are the dangers inherent in specializing in software first at the expense of the other two?
2. Discuss the nature of a **z-score** in statistics. Specifically, what does the ratio $x - \mu$ to $\sigma$ tell us? Second, will standardizing a raw distribution make the distribution normal? Why or why not? Explain.
3. The **arithmetic mean** is a common measure of **central tendency**. But what is it exactly? Describe in words what the arithmetic mean actually accomplishes. Look at the formula and "unpack" its meaning as much as you can. How would you describe it to someone?
4. Describe in words what a **standard deviation** is by unpacking the formula for it in some detail. Explain also why it is necessary to square deviations. What would be

the consequence of leaving them **unsquared**? What is the square of the standard deviation called and what does it accomplish?

5. For an increasingly **large sample size**, what is the consequence of dividing by $n$ rather than $n-1$ in the standard deviation? Play with some numbers and allow $n$ to get extremely large. What happens as $n$ goes toward infinity?
6. Take a good look at the function rule for the **normal density**. What does the formula reduce to for a standardized $z$-distribution? Perform the relevant substitutions to obtain the simplified form of the distribution.
7. Distinguish between a **vector** and a **matrix** in linear algebra.
8. Distinguish between a **covariance** vs. a **correlation** matrix. How is one the standardized counterpart of the other? Explain.
9. Why is the **upper triangular** the same as the **lower triangular** in a covariance or correlation matrix? Draft a sample matrix with numbers to demonstrate.
10. An **identity matrix** is one in which there are values of 1.0 along the main diagonal of the matrix from top left to bottom right, and values of 0.0 everywhere else. Construct an identity matrix in Python of dimension $3 \times 3$.
11. Discuss the nature of the equation $\mathbf{Ax} = \lambda\mathbf{x}$. In words, what is it communicating? Draw a figure depicting its main features.
12. Recall the **quadratic equation** from high school and featured in the chapter. What does the equation actually accomplish? In other words, what does it mean to calculate the "**roots of an equation**" and when is the quadratic equation necessary for such computation?